

XLeRobot Digital Twin — Specification & Plan

Course: RBB2013 Digital Twin (May 2026)

Repository: <https://github.com/ManHazz/RBB2013-Digital-Twin>

Team members:

Name	Student ID
Muhammad Aiman bin Ahmad Hazimin	22011708
Hazieq Danial bin Roshihan Annuar	24006633
Muhammad Raziq bin Sufian	24006626
Ibrohim bin Ahmad Jaafar Sadzik	24006396
Ariq Danish bin Nor Razak	24006796

Rubric: Project Specification & Plan (5%)

1. Problem statement

Robot arms in industrial and lab settings are typically driven by low-level joint commands scripted by engineers. This makes them:

- **Inaccessible** to non-programmer operators who understand the *task* but not the joint math.
- **Slow to iterate** — every new task requires new joint scripts, new safety checks, new visual verification.
- **Opaque during operation** — no live insight into the arm's actual joint state, end-effector pose, or task progress.

The XLeRobot Digital Twin addresses all three.

2. Purpose and SMART outcome

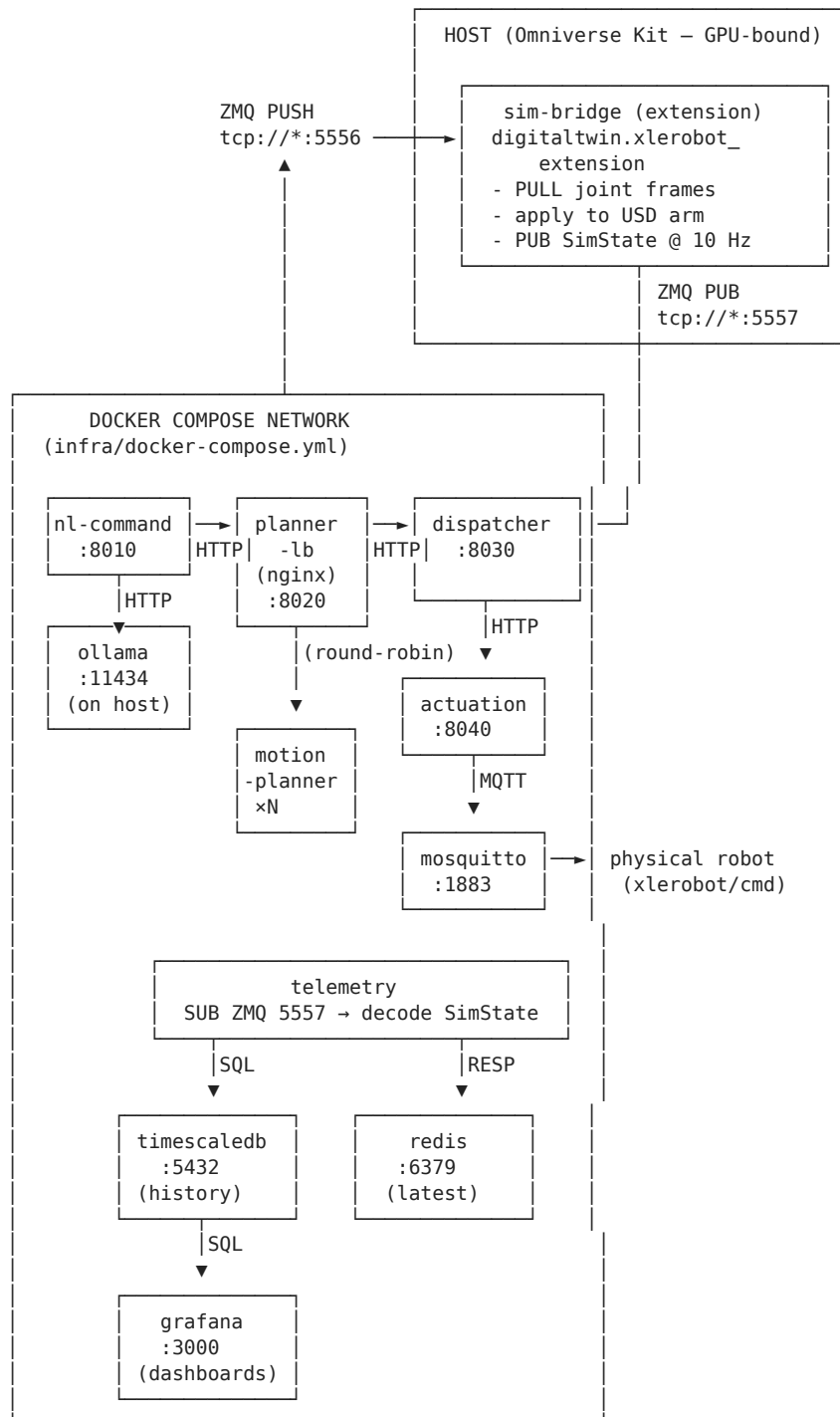
A digital twin of a 6-DoF robot arm that lets a non-programmer operator command the physical robot in **natural language**, verify the motion **in simulation before execution**, and observe **live telemetry** of every attempted move.

SMART outcome: Given a user's plain-English command (e.g. "pick up the ball"), the digital twin shall

- **Specific** — resolve the command to a target end-effector pose using an LLM,
- **Measurable** — produce 6 joint angles within $\pm 1 \times 10^{-3}$ radian of the target's inverse kinematics solution (verified in `tests/unit/test_motion_planner.py`),
- **Achievable** — stream the interpolated motion to an Omniverse simulation at 30 fps,
- **Relevant** — publish the validated joint command to the physical robot over MQTT topic `xlerobot/cmd`,
- **Time-bound** — complete the full loop (command → sim frame received → row in TimescaleDB) within **10 seconds**, verified by an automated system test (`tests/system/test_end_to_end.py`).

Success is measured by the automated system test passing on every push.

3. Block diagram



4. Function of each module

#	Module	Was	Function	Runtime
1	nl-command	llm_controller.py	Take user text → call Ollama LLM → parse action steps → return TargetPose	container :8010
2	motion-planner	robot_ik.py	Inverse kinematics + reachability + collision check → return 6 joint angles + flags	container :8020
3	planner-lb	new (nginx)	Round-robin POST /plan across N motion-planner replicas (horizontal scale)	container :8020
4	dispatcher	interpolator	30 fps interpolation from current to target joints, ZMQ PUSH each frame to sim, HTTP-trigger actuation	container :8030 + ZMQ :5556
5	sim-bridge	extension.py	Apply joint angles in Omniverse USD stage, PUB live SimState at 10 Hz	HOST (Omniverse Kit) — cannot containerize (GPU + Kit runtime)
6	actuation	MQTT publisher	On dispatcher trigger, publish xlerobot/cmd MQTT message to physical robot	container :8040
7	telemetry	new	SUB sim state → write one Timescale row per tick + overwrite Redis state:latest	container (daemon)
—	ollama	infra	Local LLM inference (Qwen2.5 3B)	host :11434
—	timescaledb	infra	Time-series persistence — hypertable robot_state	container :5432
—	redis	infra	Latest-state cache — key state:latest, AOF+RDB persistence	container :6379
—	mosquitto	infra	MQTT broker for outbound-to-robot commands	container :1883
—	grafana	infra	Visualization from TimescaleDB — provisioned dashboard XLeRobot — Robot State	container :3000

Why sim-bridge lives on the host: Omniverse Kit needs GPU-accelerated Vulkan + X server + its own bundled Python. Containerizing would need GPU passthrough + X11 forwarding for negligible benefit. The boundary is deliberate and documented (see docs/ARCHITECTURE.md §3).

5. Data and protocols between every pair of modules

This is the normative table — every pair of communicating modules, with route/topic, port, protocol, data format, when the communication is initiated, when it is concluded.

From → To	Route/Topic	Port	Protocol	Data format	Initiated	Concluded
client → nl-command	POST /command	8010	HTTP/JSON	{text} → {x,y,z}	user submits text	pose returned
nl-command → ollama	POST /api/generate	11434	HTTP/JSON	prompt → completion	on each command	completion returned
nl-command → motion-planner	POST /plan	8020	HTTP/JSON	{target} → {joints[6]}, reachable, collision_free	pose resolved	plan returned
motion-planner → dispatcher	POST /dispatch	8030	HTTP/JSON	{joints[6]} → {accepted}	plan valid	ack
dispatcher → sim-bridge	joint frames	5556	ZMQ PUSH/PULL	{joints[6], frame_id}	dispatch accepted	last (30th) frame sent
sim-bridge → telemetry	sim state	5557	ZMQ PUB/SUB	{joints[6], ee_pose, target, obstacles, ts}	every sim tick (10 Hz)	run ends
telemetry → timescaledb	INSERT	5432	SQL	hypertable row robot_state	on each state msg	commit
telemetry → redis	SET	6379	RESP	key state:latest, JSON value	on each state msg	overwritten by next
dispatcher → actuation	POST /actuate	8040	HTTP/JSON	{joints[6]} → {published, topic}	after last sim frame sent	MQTT rc = 0
actuation → mosquito	xlerobot/cmd	1883	MQTT/JSON	{joints[6]}	run validated in sim	broker ack
grafana → timescaledb	SELECT	5432	SQL	time-series read	dashboard refresh (5 s)	rows returned

Per-pair payload examples, initiation conditions, conclusion conditions, and error modes live in `contracts/interface-contracts.md` (one section per pair, owner-tagged, filled in during sprint 1).

```
In [ ]: # The single source of truth for data types exchanged between modules.
# Frozen at v1.0 in sprint 1. Only the tech lead edits this.

from services.shared.schemas import (
    CommandRequest,    # POST /command body
    TargetPose,        # (x, y, z) end-effector target
    PlanRequest,        # POST /plan body
    PlanResponse,       # planner output: joints + reachable + collision_free
    DispatchRequest,    # POST /dispatch body
    DispatchResponse,   # dispatcher ack
    SimState,           # published by sim-bridge, ingested by telemetry
    ActuationCommand,  # MQTT payload for physical robot
)

# Every one of these is a pydantic v2 model. Every service imports from this
# module, so a contract violation at any hop fails fast with HTTP 422 rather
# than silently drifting.
```

6. Digital twin state — formal definition

At any instant t , the digital twin's state $S(t)$ is:

$$S(t) = (\text{joints}[6], \text{ee_pose}(x,y,z), \text{target}(x,y,z,r), \text{obstacles}[\dots], ts)$$

Component	Meaning	Source	Storage
<code>joints[6]</code>	6 joint angles (rad)	Omniverse sim tick	Redis <code>state:latest</code> + Timescale <code>robot_state.joints</code>
<code>ee_pose(x,y,z)</code>	End-effector position (cm)	Forward kinematics on joints	Same
<code>target(x,y,z,r)</code>	Scene target ball	<code>extension.py</code> <code>TARGET_BALL</code>	Same
<code>obstacles[...]</code>	Named obstacle spheres	<code>extension.py</code> <code>OBSTACLES</code>	Same
<code>ts</code>	Wall clock timestamp	Publisher	Timescale partition key

Latest state — Redis key `state:latest`, overwritten every ~100 ms. O(1) reads for "where is the arm now?".

Historical state — TimescaleDB hypertable `robot_state`, one row per tick. Cheap time-range queries for Grafana panels and post-hoc analysis.

Both storages are persistent (named Docker volumes) and survive `docker compose restart`. See `docs/PERSISTENCE_PROOF.md` for evidence.

7. Real data streams and aggregation

Four concurrent real streams (no mocked inputs in the live system):

#	Stream	Producer	Transport	Rate
1	User commands	External <code>curl</code> /UI	HTTP POST / command	on-demand
2	LLM completions	Ollama	HTTP JSON	per-command
3	Interpolated joint frames	<code>dispatcher</code>	ZMQ PUSH	30 fps × ~1 s per command
4	Sim state	<code>sim-bridge</code> (Kit extension)	ZMQ PUB	10 Hz continuous

Aggregation happens in `telemetry`: subscribes to stream #4, decodes into `SimState`, then dual-writes:

- One row per message → TimescaleDB (historical aggregation)
- Overwrite `state:latest` key → Redis (latest aggregation)

See `docs/DATA_STREAMING.md` for the full aggregation mechanics.

8. Visualization and measure of success

Two complementary visualizations prove the digital twin works end-to-end.

8.1 Omniverse Kit viewport (the "physical twin" view)

3D scene rendered at 30+ FPS with:

- 6-DoF robot arm (jointed, materials, gripper)
- Red target ball
- Two purple obstacle spheres

The arm animates in real time as joint commands stream in from `dispatcher` via ZMQ PUSH. **Visual measure of success:** the gripper reaches and touches the red ball when the user asks it to "pick up the ball".

8.2 Grafana dashboard `XLeRobot - Robot State` (the "data twin" view)

Panels sourced live from TimescaleDB:

- **End-effector position over time** — three traces (`ee_x`, `ee_y`, `ee_z`).
- **Joint 0 (shoulder rotation)** — trace over the same window.
- **Rows in last 5 minutes** — stat panel proving persistent write throughput.

Data measure of success: the `ee_x/ee_y/ee_z` traces converge on the ball's coordinates `(40, 1.75, 0)` cm by the end of a run, and the row-count stat is strictly increasing during a run.

Screenshots + interpretation live in `docs/VISUALIZATION.md`.

9. Plan of execution (2 sprints)

Sprint 1 (26–30 Jul 2026) — tag `sprint-1`

- Scaffold repo, freeze pydantic v2 contracts, extract each subsystem into its own FastAPI service.
- Per-service unit tests with **pass AND fail cases** (empty command → 422, unreachable target → `reachable=false`, colliding target → `collision_free=false`).
- 10 pair-wise interface contracts documented.
- **Deliverables merged and tagged.**

Sprint 2 (31 Jul 2026) — tag `sprint-2`

- `infra/docker-compose.yml` wiring all services + infra.
- Integration tests per service pair (real Timescale, Redis, mosquito via testcontainers).
- System test end-to-end (`command` → row in Timescale within 10 s).
- `.github/workflows/ci.yml` — lint → unit → integration → regression on every push.
- Grafana provisioning + dashboard.
- Persistence proof — restart Timescale + Redis, data + state survive.
- Horizontal scale demo — `motion-planner` × 3 behind nginx round-robin.

Per-teammate atomic tasks + ownership in `TASK_ALLOCATION.md`. What actually shipped each sprint in `docs/SPRINT_LOG.md`.

10. Repository layout

```
xlerobot/
├── services/
│   ├── nl_command/          # LLM intent parser (FastAPI :8010)
│   ├── motion_planner/     # IK + collision (FastAPI :8020)
│   ├── dispatcher/         # interpolator + ZMQ PUSH (FastAPI :8030)
│   ├── actuation/          # MQTT publisher (FastAPI :8040)
│   ├── telemetry/          # ZMQ SUB → Timescale + Redis (daemon)
│   └── shared/schemas.py   # frozen pydantic v2 contract types v1.0
├── sim/
│   └── extension.py         # host-only pointer stub → Kit extension
├── kit-app-template/       # Omniverse Kit app + digitaltwin.xlerobot_extension
├── infra/
│   ├── docker-compose.yml
│   ├── mosquitto/mosquitto.conf
│   ├── timescaledb/init.sql
│   ├── nginx/nginx.conf    # planner-lb round-robin
│   └── grafana/             # provisioning + dashboard JSON
├── tests/
│   ├── unit/                # per-service pure logic
│   ├── integration/         # service pairs vs real infra
│   ├── system/test_end_to_end.py # full stack assertion
│   ├── regression/test_golden_ik.py # golden IK cases in CI
│   └── contracts/interface-contracts.md # normative per-pair contracts
├── docs/
│   ├── SPECIFICATION.ipynb   # this file
│   ├── ARCHITECTURE.md
│   ├── AI_MODEL.md
│   ├── DATA_STREAMING.md
│   ├── VISUALIZATION.md
│   ├── SCALING_PROOF.md
│   ├── PERSISTENCE_PROOF.md
│   ├── DEMO.md
│   └── SPRINT_LOG.md
├── .github/workflows/ci.yml   # lint + unit + integration + regression
├── PLAN.md
├── TASK_ALLOCATION.md
└── README.md
```

11. Summary

The XLeRobot Digital Twin turns natural language into validated, collision-free motion of a 6-DoF arm, with every attempted move captured, visualized, and made persistent. It composes an LLM (Qwen2.5) for intent, a classical IK+collision solver for motion, an Omniverse simulation for the physical twin, and a TimescaleDB+Redis+Grafana stack for the data twin — all containerized (except the sim, which is host-bound for correct reasons), all documented per interface pair, and all continuously tested from unit through integration through system tiers.

Success is not a static claim; it is proven by the CI-gated automated tests, the compose stack that comes up in one command, and the two visualizations that make both physical and data state observable at all times.